High Dimensional Data Analysis Project: Recommendation Systems via Approximate Matrix Factorization

Project Report
Group ID: 2

Danil Afonchikov
Daniel Chukwuemeka David
Kelvin Asu Ekuri

1 Introduction

Recommendation systems play a crucial role in modern digital platforms, enabling personalized content and product suggestions. This project explores matrix factorization techniques for collaborative filtering, aiming to predict user preferences by decomposing a sparse preference matrix into lower-dimensional feature matrices.

1.1 Motivation

The economic potential of recommendation systems is immense. As illustrated by the Netflix Prize competition in 2007, where a $1 million reward was offered for improving recommendation algorithms by 10%, these systems directly impact user engagement and business revenue.

2 Mathematical Model: Matrix Factorization

2.1 Problem Formulation

Consider a preference matrix $X \in R^{m \times n}$, where m represents users and n represents items. The goal is to approximate X using a low-rank factorization:

$$X \approx UV^T \tag{1}$$

Where:

- $U \in R^{m \times r}$ represents user feature matrix
- $V \in R^{n \times r}$ represents item feature matrix
- r is the rank (number of latent features)

2.2 Objective Function

The optimization problem is formulated as:

$$\min_{U,V} \sum_{(i,j) \in \Omega} W_{ij}(X_{ij} - [UV^T]_{ij})^2 \tag{2}$$

Where:

- Ω denotes known entries
- W_{ij} is a weight matrix (1 for known entries, 0 for unknown)

3 Implemented Methods

3.1 Gradient Descent (GD)

Gradient Descent is straightforward to implement but suffers from slow convergence due to its uniform step size. The update rules are:

$$U \leftarrow U - \alpha \nabla_U J \tag{3}$$
$$V \leftarrow V - \alpha \nabla_V J \tag{4}$$

Key hyperparameters:

- Learning rate α: A high α causes divergence, while a low α leads to slow convergence.
- Convergence tolerance: Determines when updates cease.

Performance:

- Final RMSE: 0.1802
- Convergence: 62 iterations
- Limitation: Sensitive to α, requiring careful tuning.

3.2 Block Coordinate Descent (BCD)

BCD alternately updates U and V, which leads to faster convergence compared to GD. It systematically minimizes the loss by solving subproblems for U and V:

1. Fix V, solve $U = \arg\min_U J(U, V)$
2. Fix U, solve $V = \arg\min_V J(U, V)$

Key hyperparameters:

- Number of iterations: Determines convergence.
- Initial values: Poor initialization can lead to suboptimal solutions.

Performance:

- Final RMSE: 0.1489
- Convergence: 35 iterations

3.3 Accelerated Gradient Descent (AGD)

AGD introduces momentum to accelerate convergence, especially for smooth optimization problems. The update rules include:

$$Y_U = U + \beta(U - U_{prev}) \tag{5}$$
$$U_{new} = Y_U - \alpha \nabla J(Y_U) \tag{6}$$

Key hyperparameters:

- Momentum β: Larger values speed convergence but may overshoot optima.
- Gradient clipping: Prevents exploding gradients.

Performance:

- Final RMSE: 0.1611
- Convergence: 40 iterations

3.4 Alternating Least Squares (ALS)

ALS solves the matrix factorization problem by leveraging the closed-form solution for each subproblem:

$$U = (V^T W V + \lambda I)^{-1} V^T W X \tag{7}$$
$$V = (U^T W U + \lambda I)^{-1} U^T W X^T \tag{8}$$

Key hyperparameters:

- Regularization λ: Balances overfitting and underfitting.

Performance:

- Final RMSE: 0.1488
- Convergence: 41 iterations
- Observation: Most efficient method due to direct solutions.

3.5 Weighted Singular Value Decomposition (Weighted SVD)

Weighted SVD explicitly incorporates the weight matrix, making it more flexible:

$$\min_{U,V} \|W \odot (X - UV^T)\|_F^2 + \lambda(\|U\|_F^2 + \|V\|_F^2) \tag{9}$$

Performance:

- Final RMSE: 0.1487
- Convergence: 40 iterations

3.6 Alternating Direction Method of Multipliers (ADMM)

ADMM splits the problem into subproblems with augmented Lagrangian methods for handling constraints and noise:

$$L(U, V, Z) = \|W \odot (X - UV^T)\|_F^2 + \lambda(\|Z\|_F^2 + \rho\|Z - UV^T\|_F^2) \tag{10}$$

Performance:

- Final RMSE: 0.5460
- Convergence: 41 iterations
- Observation: High computational complexity and slower convergence.

4 Experimental Results

4.1 Performance Metrics

Root Mean Square Error (RMSE) was used to evaluate model performance:

$$\text{RMSE} = \sqrt{\frac{\sum_{(i,j)\in\Omega} W_{ij}(X_{ij} - [UV^T]_{ij})^2}{\sum W_{ij}}} \tag{11}$$

4.2 Hyperparameter Influence

Hyperparameter selection significantly impacts the performance and convergence of matrix factorization techniques. This section highlights key observations from experiments with various hyperparameter values.

4.2.1 Rank (r)

The rank r determines the number of latent features in the factorization. It directly impacts the model's ability to capture patterns in the data:

For this project, the rank of the matrix was kept at $r = 10$ for all implementations, and the regularization parameter was fixed at $\lambda_{\text{reg}} = 0.01$. This ensured a consistent baseline for comparing the algorithms' performance.

Observations:

- Increasing r typically reduces RMSE, as higher ranks capture more latent features.
- Excessive increases in r (e.g., $r > 15$) caused some algorithms to behave erratically, likely due to overfitting or computational instability.
- Through grid search, $r = 10$ emerged as the ideal choice for most implementations, offering a good balance of performance and stability. A rank of $r = 15$ also performed adequately in some cases, but higher ranks generally led to suboptimal behavior.

4.2.2 Regularization Parameter (λ_{reg})

Regularization helps control overfitting by penalizing large values in the factor matrices:

- $\lambda_{\text{reg}} = 0.001$: Fast convergence but higher risk of overfitting.
- $\lambda_{\text{reg}} = 0.01$: Balanced trade-off between convergence speed and generalization.
- $\lambda_{\text{reg}} = 0.1$: Slower convergence but better generalization and reduced overfitting.

4.2.3 Momentum Parameter (β)

Momentum accelerates convergence by incorporating information from previous updates:

- $\beta = 0.5$: Moderate convergence speed but less effective for larger datasets.
- $\beta = 0.9$: Faster convergence and smoother optimization, especially for smooth loss surfaces.

4.2.4 Gradient Clipping

Gradient clipping prevents exploding gradients during optimization:

- gradient clip = 1.0: Effective for stabilizing optimization with moderate learning rates.
- gradient clip = 5.0: Particularly useful with high learning rates, avoiding overshooting.

4.3 Impact on Methods

The choice of these hyperparameters varies between methods:

- Gradient descent (GD): Sensitive to α, which requires careful selection to avoid divergence.
- Accelerated Gradient Descent (AGD): Performance improves significantly with higher β but requires gradient clipping to prevent instability.
- Alternating Least Squares (ALS): Relies more on λ_{reg} for generalization than on other hyperparameters.
- Weighted SVD: Sensitive to λ_{reg} and benefits from moderate regularization.

4.4 Comparative Analysis

Method	Final RMSE	Convergence (Iterations)
Gradient Descent (GD)	0.1802	62
Block Coordinate Descent (BCD)	0.1489	35
Accelerated Gradient Descent (AGD)	0.1611	40
Alternating Least Squares (ALS)	0.1488	41
Weighted SVD	0.1487	40
ADMM	0.5460	41

5 Conclusion

Matrix factorization methods exhibit varying strengths. ALS and Weighted SVD outperform others in RMSE and convergence speed, while ADMM provides flexibility for noisy data but at a computational cost. Proper hyperparameter tuning is critical to optimizing performance.